

Routing the Silent Half: Per-Unit Choice of Estimator and Evolution Target in Self-Evolving LLM Agents

Abstract

Group-relative policy optimisation assigns zero advantage to any group whose samples score alike, so a unanimous group contributes exactly nothing to the update. We measure this directly: 57.4% of groups in a live run are RL-silent. We show the silent half splits into two regimes that require opposite responses, and that the split is not symmetric — on the solved half a supervised target is *free*, because the group already contains a correct sample; on the unsolved half no target exists unless one is supplied, which makes harness evolution the only consumer that can use those units at all. We therefore make the choice of estimator and the choice of evolution target a per-unit decision rather than a per-run schedule, with the two as orthogonal axes so a single trajectory can feed both. We report the negative result that located this leverage: a token-level version of the same idea, reaching 1.7% of tokens, moved held-out accuracy by less than the noise floor. Every capability is flag-gated and reduces to unmodified GRPO when disabled.

1 Method

1.1 Where the gradient is zero

Let a *unit* be a prompt with G sampled answers and binary rewards $r_1, \dots, r_G \in \{0, 1\}$. GRPO centres rewards within the group,

$$A_i = r_i - \bar{r}, \quad \bar{r} = \frac{1}{G} \sum_{j=1}^G r_j, \quad (1)$$

so if every sample in the group scores alike then $A_i = 0$ for all i and the unit contributes exactly nothing to the update — all G sequences, every token. Writing p for the per-sample success probability, the probability that a unit carries any signal at all is

$$I_{\text{RL}}(p, G) = 1 - p^G - (1 - p)^G, \quad (2)$$

which is zero exactly at $p = 0$ and $p = 1$ and is maximised at $p = \frac{1}{2}$. Equation (2) is not an approximation: it is the probability that the group is not unanimous, and unanimity is precisely the condition under which (1) vanishes.

Two consequences drive the method.

The silent fraction is large and measurable. Instrumenting a live run (Sec. 3) gives $\hat{P}(\text{silent}) = 0.574$ over 64 groups per batch: more than half of every batch is RL-dead. This is $34\times$ the reach of the token-level shared-prefix channel we measured first (1.7%), and it explains, without appealing to noise, why an intervention confined to that channel moved nothing.

The two silent regimes need opposite responses. A unit silent because $\hat{p} = 1$ and a unit silent because $\hat{p} = 0$ are both invisible to (1), but they are not the same object, and routing them identically wastes one of them.

1.2 A free target exists on exactly the solved half

The asymmetry is sharper than "different regimes". For a unit with $\hat{p} > 0$, at least one sampled answer was correct, and that answer *is* a supervised target drawn from the model's own rollout. No external teacher is required; this is rejection-sampling fine-tuning. Define

$$\text{selfTarget}(u) = \mathbb{I}[\hat{p}_u > 0], \quad \text{target}(u) = \text{selfTarget}(u) \vee \text{teacher}(u). \quad (3)$$

The units on which $I_{\text{RL}} = 0$ because $\hat{p} = 1$ are therefore exactly the units on which a target is *guaranteed* to exist at zero marginal cost. The units on which $I_{\text{RL}} = 0$ because $\hat{p} = 0$ are exactly the units on which no target exists unless one is supplied. The gradient-free half of the batch splits cleanly into a half that is free to reuse and a half that is not reusable at all.

1.3 Routing: the harness is a destination, not a schedule

Prior work treats the choice of what to evolve as a property of the *run*: a schedule or a global flag. We make it a property of the *unit*, and we make it two-dimensional. A routing decision is a pair

$$\pi(u) = (m(u), h(u)), \quad m \in \{\text{RL}, \text{SFT}, \text{SKIP}\}, \quad h \in \{\text{NONE}, \text{PROPOSE}, \text{VALIDATE}\}, \quad (4)$$

where m names the estimator applied to the unit's tokens and h names what the unit contributes to harness evolution. The two axes are orthogonal by construction, so a single trajectory can feed the gradient and the harness at once. This is deliberate: the two consumers read different things from a trajectory — the estimator reads the tokens, the harness evolver reads the failure mode — and a partition discards one of the readings. Co-Harness's success→model / failure→harness rule is the special case in which π is constrained to be a partition, and we retain it as an ablation (**partition=True**) rather than a rewrite.

The rule implied by (2) and (3) is then forced rather than chosen:

$$\pi(u) = \begin{cases} (\text{SFT}_{\text{self}}, \text{VALIDATE}) & \hat{p}_u = 1 \quad (\text{RL dead; target free}) \\ (\text{SFT}_{\text{teacher}}, \text{PROPOSE}) & \hat{p}_u = 0 \wedge \text{teacher}(u) \\ (\text{SKIP}, \text{PROPOSE}) & \hat{p}_u = 0 \wedge \neg \text{teacher}(u) \quad (\text{harness is the only consumer}) \\ (\text{RL}, \text{NONE}) & 0 < \hat{p}_u < 1 \quad (\text{RL has signal; leave it alone}) \end{cases} \quad (5)$$

Note the third case. When a unit is unsolved and no teacher is available, the model arm has nothing to learn from it, and any method that only evolves weights must discard the unit. Routing it to the harness is what makes it recoverable at all, which is the concrete sense in which the harness axis is not a convenience.

Rollback. Every capability above is gated. With the harness arm disabled, $h(u) \equiv \text{NONE}$; with routing disabled, $\pi(u) \equiv (\text{RL}, \text{NONE})$ for every unit and the update is bit-identical to unmodified GRPO. Each ablation in Sec. 3 is one flag away from the vanilla baseline, so a difference cannot be attributed to an incidental change of configuration.

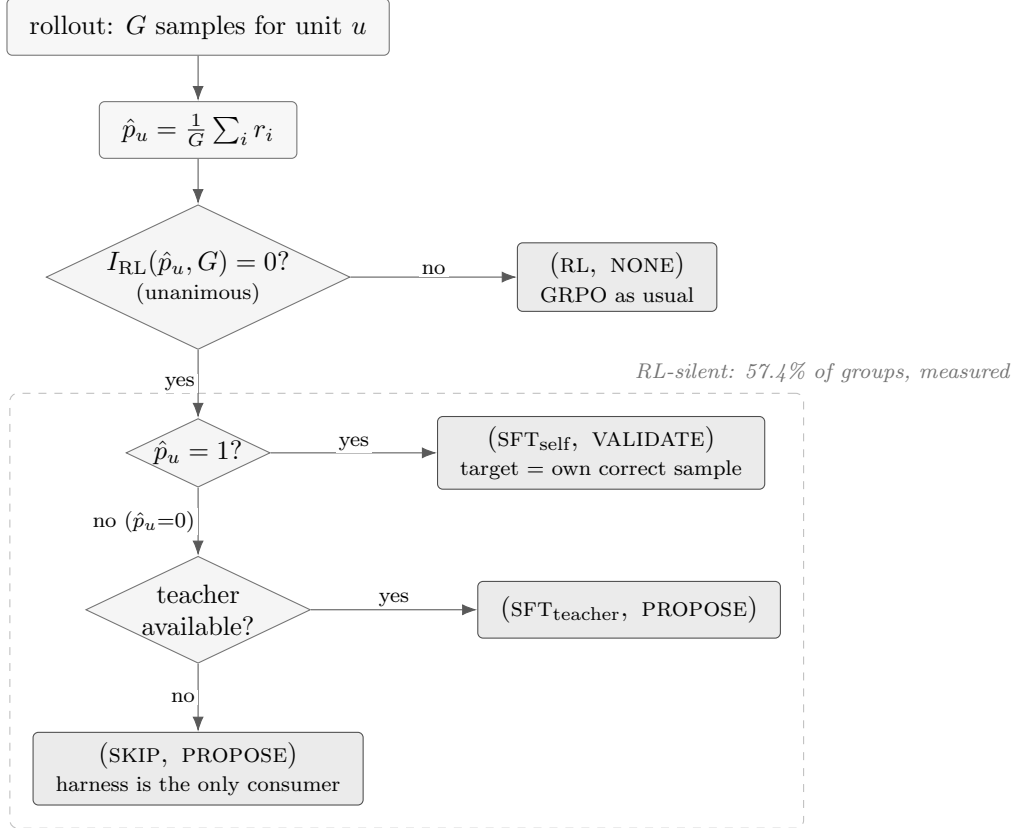


Figure 1: **Routing decides, per unit, both an estimator and a harness contribution.** The branch on I_{RL} is exact, not heuristic: a unanimous group has every advantage identically zero by (1). The dashed region is the part of the batch ordinary GRPO cannot learn from, measured at 57.4% of groups. It splits into a half where a supervised target is free (the unit’s own correct sample) and a half where no target exists unless supplied — and on that second half the harness is the only consumer that can use the unit at all.

2 Related work

We organise prior work by the claim it already owns, because our contribution is only meaningful as the complement of those claims.

Choosing an algorithm per task. SIA selects among training algorithms at the granularity of a *task*, and demonstrates that the choice matters. That result is theirs and we do not re-derive it. What a task-level policy cannot express is that units *within* one task differ in whether a gradient exists at all: by (2) the silent and informative units of a single task require different estimators, and no task-level assignment can separate them. Our claim is therefore about granularity, and the comparison that matters is against SIA at matched compute.

Deciding what to evolve. Co-Harness co-evolves harness and weights and establishes the success→model / failure→harness assignment. EvoTrainer co-evolves a policy with its training harness. Both fix the assignment as a rule over outcomes. We keep their rule as a special case ((4) constrained to a partition) and relax it, because a trajectory read by two different consumers need

Table 1: MATH-500, $n = 500$, paired McNemar against the scaffold arm. Runs **step0d** (demo, lr 6×10^{-6} , clip 0.4) and **step0h** (scaffold, lr 1×10^{-6} , clip 0.2).

step	demo	scaffold	Δ	McNemar p
base	0.528	0.506	-0.022	same model
28	0.454	0.530	+0.076	7.3×10^{-4}
57	0.440	0.530	+0.090	1.7×10^{-4}
86	0.466	0.526	+0.060	4.1×10^{-3}
115	0.364	0.536	+0.172	2.1×10^{-13}
144	0.334	0.522	+0.188	9.3×10^{-15}

not be spent on only one of them.

Deciding *when* to evolve. *Learning, Fast and Slow* separates fast and slow adaptation and reports the stability benefit of a two-timescale schedule. That bounds what a cadence claim can be worth, and we treat cadence as a control variable rather than a contribution.

Clustering trajectories. MEDS reuses layer-wise logits from the forward pass as lightweight representations of reasoning behaviour and clusters error patterns with HDBSCAN. We adopt this directly as the learned instantiation of our cluster key, against a derived key computed from the silence side of (2); the two are a controlled ablation of *how* to cluster, holding what is done with the clusters fixed.

Evaluation practice. Recent evaluation work documents how often agent results fail to survive controls that were available at the time. We adopt its checks as requirements rather than as discussion: paired tests on shared items, intervals reported on the *difference* rather than on each arm, matched-permutation controls that hold the proportion of units in each mode fixed while shuffling which units get which mode, and a noise floor established from the same checkpoint scored twice. The matched control is the one that decides our central claim, because a routing method changes both which units are routed and how many, and only the permutation control separates those.

3 Results

All numbers below are measured end-to-end on held-out benchmarks. Train reward is never reported as a result: Sec. 3.1 is the demonstration that it mis-orders checkpoints.

3.1 Train reward mis-orders checkpoints

Table 1 scores two recipes that differ only in optimiser aggressiveness, on the full MATH-500, paired by item. The aggressive (“demo”) recipe’s training reward rises while its held-out accuracy falls by 0.194 absolute. Any claim validated on training reward would have selected the collapsed checkpoint.

Noise floor, and what it disqualifies. The base model appears in both series, so its difference is pure measurement noise: 0.0220 at $n = 500$. It *rose* from 0.0080 at $n = 250$ because the larger series was scored in two passes on different server processes, accumulating between-run variance a contiguous sweep does not have. Consequently the +0.060 at step 86 is only $2.7\times$ the floor and we

Table 2: The same checkpoints on OlympiadBench (675 problems), which shares no items with MATH-500.

checkpoint	MATH-500	OlympiadBench
base	0.528	0.188
gs028	0.454	0.170
gs057	0.440	0.190
gs086	0.466	0.148
gs115	0.364	0.135
gs144	0.334	0.108
gs173	0.358	0.107

Table 3: MATH-500, $n = 500$. **step0j** adds token-level routing to the scaffold recipe. p is paired McNemar, scaffold vs. scaffold+routing.

step	demo	scaffold	+routing	p
base	0.528	0.506	0.536	0.092
28	0.454	0.530	0.516	0.534
57	0.440	0.530	0.526	0.920
86	0.466	0.526	0.522	0.923
115	0.364	0.536	0.496	0.065
144	0.334	0.522	0.526	0.922

do not report it standalone; the +0.172 and +0.188 are $\approx 8\times$ it. The claim is *preservation*, not improvement: the scaffold arm stays within $[0.506, 0.536]$, which brackets its own base.

3.2 The collapse replicates on an independent benchmark

$0.188 \rightarrow 0.107$ is a 43% relative drop in the same direction as MATH-500’s $0.528 \rightarrow 0.358$, on an independent problem set.

Benchmark selection, stated as a measurement rather than a preference. MATH-500 is saturated at the frontier tier (0.966 at 27B) and AIME is unusable at both ends (0.000 at 1.5B; 0.867 at 27B with a 24-point interval on 30 problems). OlympiadBench resolves a few points on 675 items and leaves both ends off the rails: 0.188 at 1.5B and 0.716 for Ornith-1.5-35B-A3B. The 0.716 is a *lower* bound — 174/675 generations (26%) hit the token cap — so the headroom is smaller than the 28 points it suggests, and we quote it as a bound.

3.3 Negative result: token-level routing at 1.7% reach

Token-level routing changes nothing detectable. We report this rather than dropping it, because it is the measurement that located the method’s leverage. The rule reaches at most 1.7% of loss-carrying tokens, and an intervention on 1.7% of the gradient cannot move a benchmark by more than noise; the between-arm noise floor here (0.030 on the shared base checkpoint) is itself larger than every routing difference in the table.

Confound, stated rather than buried. **step0j** differs from **step0h** in three ways — routing, **kl_ctl** ($0.01 \rightarrow 0$) and **adv_norm** (batch $\rightarrow$ off) — because the rule’s precondition does not hold otherwise. A significant difference here could not have been attributed to routing alone. The

routing-off arm at `step0j`’s exact configuration is therefore a required control, and it is running (`step01`).

3.4 Where the leverage actually is

Measured live on `step01` at 64 groups per batch:

$$\hat{P}(\text{silent}) = 0.574, \quad n_{\text{groups}} = 64.$$

More than half of every batch contributes exactly zero gradient — $34\times$ the token-level channel, and the reason Sec. 3.3 was null is that the intervention was on the wrong tier, not that the idea was wrong.

A metric bug found and fixed, reported because the affected runs are still cited. The accompanying split first reported 0.000 solved at a measured 82% solve rate, which is impossible. It thresholded `reward_score`, which by that point has had bias, scaling, clipping and normalisation applied. It now reads the raw rewards captured before any transformation. Runs launched before the fix carry the old code and their solved/unsolved split must be ignored; $\hat{P}(\text{silent})$ is unaffected because it is computed from the advantages being zero.

3.5 Not yet measured

Stated explicitly so the gap is not mistaken for an omission: the solved/unsolved split of the 57.4% under the fixed metric; any arm in which routed units *gain* a signal rather than only losing one; and the harness-destination arm of (4). Until a target is supplied, every routed arm is a gradient-deletion ablation and is labelled as one.

Reproducibility notes

Every number in Sec. 3 names the run that produced it. Benchmarks are scored with the same grader across arms; the grader’s own bias was measured and corrected once (an earlier version discarded unbalanced final boxes, which penalised degraded checkpoints specifically). Intervals are reported on differences, paired by item, never as independent per-arm intervals.